

Highlights

A Physics-Informed Neural Network Forward Solver for the Rough-Volatility Calibration Inverse Problem: Dimension Scaling on the Markovian-Lifted PDE and a Fast Warm-Start Hybrid

Aviral Sharma, Kamal Preet Singh, Vipul Sharma

- PINN forward solver for the Markovian-lifted rough-Heston pricing PDE
- Solver error shows no detectable growth in lift dimension; grid cost explodes
- Explicit decomposition separates network solve error from lift model error
- Parametric PINN warm-start cuts expensive forward-model calls about 9-fold
- Accuracy limits mapped: error grows in rough, high vol-of-vol corners

A Physics-Informed Neural Network Forward Solver for the Rough-Volatility Calibration Inverse Problem: Dimension Scaling on the Markovian-Lifted PDE and a Fast Warm-Start Hybrid

Aviral Sharma^{a,*}, Kamal Preet Singh^b, Vipul Sharma^c

^a*Chandigarh University, Chandigarh, India*

^b*SGT University, Gurugram, India*

^c*Independent Researcher, India*

Abstract

Calibrating a rough-volatility model to an option surface is an inverse problem whose fractional, non-Markovian variance admits no finite-dimensional forward partial differential equation (PDE). A Markovian lift approximates the kernel by n exponentials and recovers an $(n+1)$ -dimensional forward PDE, which we solve with a physics-informed neural network (PINN) trained on its residual. Measured against an independent lifted Monte Carlo at the same n , and separated from the lift’s own approximation error, the network’s solve-error shows no statistically detectable growth as n rises from 4 to 32 (log-log slope 0.09 ± 0.08 , eight seeds), whereas a naive full-tensor finite-difference grid—an illustrative bound, not a tuned solver—is infeasible by $n=8$. The solve/lift decomposition shows whether to add factors or improve the network. A parametric extension is too inaccurate to calibrate alone, but as a warm start for a short exact-model refinement it recovers full calibration quality at 8.8 times fewer expensive evaluations. On live Bitcoin options from Deribit, the surface pins the correlation ($\rho \approx -0.43$) but identifies the Hurst exponent only to an interval, $H \in [0.04, 0.30]$, well inside the rough regime; a realised-volatility estimate using no option data falls inside it. The fit is 113 vol bp vega-weighted and 416 unweighted, the latter dominated by options under two weeks; beyond two months it is 52 vol bp.

*Corresponding author

Email address: aviral198765@gmail.com (Aviral Sharma)

Keywords: inverse problems, physics-informed neural networks, rough volatility, Markovian lift, model calibration, curse of dimensionality, scientific machine learning

1. Introduction

Model calibration is the canonical inverse problem of mathematical finance: given a finite set of noisy, arbitrage-constrained option prices, recover the parameters of a forward model that reproduces them. Rough-volatility models make this inverse problem unusually severe, because their *forward* problem is already pathological. Motivated by the empirical roughness of realised variance [1, 2] and the steep short-maturity skew that classical stochastic-volatility models such as Heston’s [3] cannot reproduce, rough-volatility models drive the variance with a fractional noise whose Hurst exponent $H < 1/2$. That single fact—whose lineage runs back to long-memory volatility [4]—makes the variance process neither Markovian nor a semimartingale, so there is no finite-dimensional forward PDE to solve, let alone invert [1, 5].

The standard engineering escape is the *Markovian lift*: approximate the fractional kernel by a sum of n exponentials and recover a genuine $(n+1)$ -dimensional forward PDE [6, 7]. This turns an ill-posed forward problem into a well-posed but high-dimensional one, and pushes the difficulty of the inverse problem onto the cost of repeatedly solving that high-dimensional forward PDE inside a calibration loop—once per candidate parameter vector, at every iteration of the outer optimiser. Any AI method proposed as a forward operator for this inverse problem must therefore be judged on two separate questions: how accurately does it solve the forward PDE it is handed, and how does that accuracy behave as the lifted dimension n —the knob that trades model fidelity against forward-problem cost—is turned up. Both questions are usually left unanswered when a neural network is dropped into a calibration pipeline as a black-box pricer.

We answer them directly for a physics-informed neural network (PINN) forward operator. Rather than laying down a grid, a PINN represents the solution as a neural network and trains it to satisfy the PDE at randomly sampled collocation points [8], sidestepping the exponential blow-up that kills finite differences in high dimensions, in the spirit of the deep Galerkin and deep BSDE solvers [9, 10]. Because it is differentiable in its inputs, the same

trained network doubles as a differentiable forward operator for gradient-based calibration once the parameters are added as additional inputs. Our concrete contributions are as follows.

1. **A convergence and stability study of the AI forward operator.** We show that the PINN’s own solve-error—the part of the total error under the network’s control—shows no statistically detectable growth as the lifted dimension climbs from 4 to 32 over eight seeds per dimension, and costs the same per training iteration at every n , while a naive full-tensor finite-difference forward solver (50^{n+1} points) is already infeasible by $n=8$. We identify the four training design choices without which this convergence behaviour does not hold (Section 4).
2. **An error decomposition that isolates what limits the inverse problem.** We split total pricing error into the network’s own solve-error (measured against a lifted Monte Carlo at the same n) and the forward model’s lift-error (measured against the exact, non-lifted Fourier forward map). Their crossover gives a concrete, maturity-dependent rule for how large n —and hence how expensive the forward operator—needs to be before the network, not the model, is the accuracy bottleneck.
3. **A differentiable forward operator embedded in the calibration inverse problem.** A parametric extension of the network, additionally conditioned on the five rough-Heston parameters, is used both as a fast stand-alone forward map and as a warm start for a short local refinement against the true forward model, and we report where its accuracy generalises and where it does not.
4. **A real-market solution of the inverse problem.** Calibrated to live Deribit Bitcoin option data, the surface identifies the correlation sharply and the Hurst exponent only to an interval that lies well inside the rough regime. We report that non-identification, its mechanism and its consequences for risk functionals, rather than a point estimate the data do not support.
5. **Fast recalibration, quantified against the true forward model.** The parametric warm start plus a short exact-model polish recovers from-scratch calibration quality at 8.8 times fewer expensive forward-model evaluations, whichever parameter box the network was trained on; we state plainly where the standalone AI forward operator is not accurate enough to trust alone.

We are explicit about what is *not* new. Papapantoleon and Rou [11] solved the lifted rough-Heston forward PDE with a mesh-free network before us. Our contribution is a residual PINN forward operator characterised specifically as a component of the calibration inverse problem: a quantitative convergence-versus-dimension study, an error decomposition that separates network error from model error, a real-market inverse-problem solution, and the warm-start hybrid.

2. Related work

Closest prior work. Papapantoleon and Rou [11] price European options in rough-diffusion models by recasting the forward pricing PDE as an energy-minimisation problem and approximating it in a time-stepping deep gradient flow, demonstrated on the lifted Heston model. Their method is the closest to ours in target and spirit; it differs in mechanism (time-stepping variational scheme versus residual PINN) and does not study forward-operator dimension scaling, error decomposition, or the calibration inverse problem downstream.

Neural forward operators for rough volatility. Jacquier and Žurič [12] cast the path-dependent pricing equations as backward SDEs and solve them with a random-feature network, reducing training to a least-squares regression. They share the forward target but not the mechanism, and neither dimension scaling nor the inverse problem is studied.

AI methods for the calibration inverse problem. The dominant approach for fast rough-volatility calibration is the supervised surrogate of Bayer et al. [13] and Horvath et al. [14]: a network trained offline on precomputed prices or implied volatilities, used as a fast forward map inside the calibration loop. It is purely data-driven; the network never sees the governing equation, carries no PDE consistency, and—because it is not derived from a physics-informed forward solve—cannot be decomposed into a network-error and a model-error term the way ours can, which is precisely the diagnostic an inverse-problem user needs to trust or distrust a calibrated fit.

PINN training pathologies and remedies. PINNs are well-known to fail as forward operators when loss terms sit on different scales or temporal causality is violated [15, 16]. The two stabilisers we use to keep the forward operator convergent and stable—self-adaptive homoscedastic-uncertainty weights [17, 18] and a causal τ -curriculum [19]—directly address these failure modes.

Foundational methods. We build on PINNs [8] and deep PDE solvers [9, 10]; on rough volatility [1, 5] and its multifactor Markovian lift [6, 7]; and on classical transform pricing [3, 20] for our independent, non-lifted forward-model ground truth.

3. Problem formulation

3.1. Rough Heston and the Markovian lift

Under rough Heston the spot price S and its instantaneous variance V satisfy

$$dS_t = \sqrt{V_t} S_t dW_t^S, \quad (1)$$

$$V_t = V_0 + \int_0^t K(t-s) \lambda(\theta - V_s) ds + \int_0^t K(t-s) \nu \sqrt{V_s} dW_s^V, \quad (2)$$

where W^S and W^V are Brownian motions with correlation ρ , while $K(t) = t^{\alpha-1}/\Gamma(\alpha)$ is the fractional kernel with $\alpha = H + \frac{1}{2}$ and $H < \frac{1}{2}$ [1, 5]. The variance is a Volterra process: non-Markovian, non-semimartingale, and admitting no finite-dimensional forward PDE. The parameter vector $(H, V_0, \theta, \lambda, \nu, \rho)$ is exactly what the calibration inverse problem must recover from market data; without a forward PDE there is no forward operator to embed inside that inverse problem at all. We fix this notation once, because the paper uses two different subsets of it: the full six-parameter vector is what the market calibration of Section 7.4 recovers, whereas the parametric forward operator of Section 7.6 conditions on the five parameters $(H, \nu, \rho, \lambda, \theta)$ that enter the PDE coefficients, holding V_0 (a state, not a coefficient) and the maturity fixed at training time.

The Markovian lift replaces K with $K(t) \approx \sum_{i=1}^n c_i e^{-x_i t}$, introducing n Ornstein–Uhlenbeck factors U^i ,

$$\begin{aligned} dU_t^i &= [-x_i U_t^i + \lambda(\theta - V_t)] dt + \nu \sqrt{V_t^+} dW_t^V, & U_0^i &= 0, \\ V_t &= V_0 + \sum_{i=1}^n c_i U_t^i, \end{aligned} \quad (3)$$

where $V_t^+ = \max(V_t, 0)$ [6, 7]. Two features of (3) drive everything that follows. First, the factors differ *only* in their mean-reversion speed x_i : they share the same drift source $\lambda(\theta - V_t)$, the same diffusion coefficient $\nu \sqrt{V_t^+}$

and, critically, the same Brownian increment dW_t^V . That common driver is why the generator in (4) below carries a full (i, j) double sum rather than a diagonal one; the derivation is given in Appendix A. Second, the speeds (x_i) span roughly ten orders of magnitude at small H , which is the source of the stiffness that dictates our numerical schemes (Section 5). The weights (c_i, x_i) are placed on a geometric quadrature grid [6]; the relative L^2 kernel error falls monotonically from 25.6% at $n=5$ to 0.39% at $n=100$, so raising n genuinely buys forward-model fidelity, at the cost of raising the forward-problem’s dimension by the same n .

3.2. The lifted forward PDE

The discounted European call price $P(t, x, u)$, where $x = \log S$ and $u = (u_1, \dots, u_n)$ collects the factor values, satisfies the $(n+1)$ -dimensional backward Kolmogorov PDE

$$\begin{aligned} 0 = & P_t + \left(r - \frac{1}{2}V\right) P_x + \frac{1}{2}V P_{xx} + \sum_{i=1}^n [-x_i u_i + \lambda(\theta - V)] P_{u_i} \\ & + \frac{1}{2}\nu^2 V \sum_{i,j=1}^n P_{u_i u_j} + \rho\nu V \sum_{i=1}^n P_{xu_i} - rP, \end{aligned} \quad (4)$$

with terminal condition $P(T, x, u) = \max(e^x - K, 0)$. Because all factors share the single Brownian motion W^V , the full double sum over (i, j) and the $(n+1)$ -dimensional cross term both appear; the PDE dimension is exactly $n+1$. At $n=4$ this is a five-dimensional forward problem; at $n=32$ it is thirty-three. Solving (4) for a single candidate parameter vector is one forward-model evaluation inside the outer calibration loop; a naive optimiser needs hundreds to thousands of these per fit (Section 6).

4. The PINN forward operator: architecture and training

4.1. Baseline network

We represent the forward map as a fully connected multilayer perceptron (MLP). Every hidden unit uses the tanh activation, which provides the smooth, high-order derivatives needed for automatic differentiation of the PDE residual; tanh is standard for PINN residual training and avoids the dead-neuron pathology of ReLU in this high-derivative setting [8]. All weight

matrices are initialised with Xavier normal initialisation [21] and all biases with zeros.

The network maps $(\tau, x, u_1, \dots, u_n) \in \mathbb{R}^{2+n}$ to a scalar correction. For the single-point forward operator (parameters fixed) we use width $W=64$ and depth $D=4$ hidden layers; for the parametric forward operator used inside the inverse problem (Section 7.6) we increase to $W=96$, $D=5$. Table 1 summarises all architectural and training hyperparameters.

4.2. Hard-constrained initial-condition ansatz

We train in time-to-maturity $\tau = T - t$ so the terminal condition becomes an initial condition at $\tau=0$. Rather than penalise it softly, we enforce it exactly through the ansatz

$$P(\tau, x, u) = P_{\text{base}}(\tau, x) + \tau N_{\theta}(\tau, x, u), \quad (5)$$

where N_{θ} is the network output. The τ prefactor forces the learned correction to vanish at $\tau=0$ automatically, so the network never fights the initial condition. Without this, a soft penalty for the terminal payoff oscillates against the PDE residual and the forward operator fails to converge.

4.3. Integrated-variance baseline anchor

The baseline P_{base} is a Black–Scholes call price. A critical design choice is what volatility it is anchored at. Anchoring at the *spot* variance V_0 leaves a systematic level bias: the factors mean-revert from V_0 toward θ over the life of the option, so the model’s actual expected variance is higher than V_0 . We anchor instead at the model’s expected *integrated* variance

$$w(\tau) = \int_0^{\tau} \mathbb{E}[V_s] ds, \quad (6)$$

computed by a short pre-flight Monte Carlo (the same simulation used for factor collocation). The baseline then uses $\sigma(\tau) = \sqrt{w(\tau)/\tau}$, which correctly captures the mean-reversion effect; the network learns only the residual stochastic-volatility correction. This single change removed a uniform ~ 120 volbp level bias we observed with the V_0 anchor—an error that would otherwise propagate directly into every inverse-problem fit using this forward operator.

Table 1: Architecture and training hyperparameters of the PINN forward operator. Values in parentheses are for the parametric extension (Section 7.6).

Component	Value
<i>Architecture</i>	
Network type	Fully connected MLP
Hidden layers	4 (5 for parametric)
Neurons per layer	64 (96 for parametric)
Activation	tanh (all hidden layers)
Weight initialisation	Xavier normal
Bias initialisation	Zero
Input dimension	$2 + n$ for single-point; $2 + n + 5$ for parametric
Output dimension	1 (price correction N_θ)
Floating-point precision	float64 (PyTorch)
<i>Optimisation</i>	
Optimiser	Adam [22]
Initial learning rate	10^{-3}
LR schedule	Cosine annealing to 0 over all iterations
Gradient clipping	ℓ_2 norm ≤ 1.0
Training iterations	10,000–20,000 (single-point); 20,000–30,000 (parametric)
<i>Collocation sampling (per iteration)</i>	
Interior collocation	2,000 uniform + 500 near-strike (total 2,500)
Boundary points	400 (split OTM / ITM)
Log-moneyness domain	$[\log K - 1.2, \log K + 1.2]$
Factor sampling	MC-empirical mean / std (Section 4.4)
Tail coverage fraction	20% of u -points drawn at $2\times$ std
<i>Loss weighting</i>	
Adaptive scheme	Learnable log-variance (homoscedastic uncertainty)
Terms balanced	PDE residual MSE, OTM boundary MSE, ITM boundary MSE
<i>τ-curriculum</i>	
Initial horizon	$\tau_0 = 0.15 T$
Ramp fraction	Linear growth to T over first 50% of iterations
<i>Compute environment (every run in this paper)</i>	
Processor	13th Gen Intel Core i5-13450HX
Logical cores	16 8
Memory (GB)	15.7
Operating system	Windows-11-10.0.26200-SP0
Python	3.12.10
PyTorch	2.12.1+cpu
PyTorch intra-op threads	10
GPU used	no

4.4. MC-informed factor collocation

The factor values u at a given maturity τ are not centred at zero: they mean-revert to a distribution set by the model dynamics. Sampling from a fixed symmetric box centred at zero causes the network to expend capacity on regions the dynamics never visit, and—critically—to under-cover the region governing the put-wing skew. Before training we run a short lifted Monte Carlo simulation (4,000 paths, 300 time steps) to estimate the mean and standard deviation of each factor at the target maturity; training collocation is then drawn from $\mathcal{N}(\hat{\mu}_i, \hat{\sigma}_i)$. An additional 20% of collocation points are drawn at twice the estimated standard deviation to extend tail coverage. This change cut smile RMSE by roughly 45% (from ~ 210 to ~ 114 volbp) relative to symmetric-box sampling.

4.5. Self-adaptive loss weighting

Three loss terms must be balanced at each training step: the PDE-residual MSE $\mathcal{L}_{\text{PDE}}$, and the MSEs at the deep-OTM ($P \rightarrow 0$) and deep-ITM ($P \rightarrow S - Ke^{-r\tau}$) boundaries. Fixed-weight balancing is brittle: the ITM boundary target is $O(1)$ while the OTM target is $O(0)$, so a single lumped weight over-emphasises the ITM term. We adopt the learnable homoscedastic-uncertainty scheme of Kendall et al. [17] in the self-adaptive spirit of McClenny and Braga-Neto [18]: each term $\mathcal{L}_k$ is assigned a learnable log-variance s_k and the objective is

$$\mathcal{L} = \sum_{k \in \{\text{PDE}, \text{OTM}, \text{ITM}\}} [e^{-s_k} \mathcal{L}_k + s_k]. \quad (7)$$

The s_k are optimised jointly with the network weights; large or noisy terms are down-weighted automatically. We never hand-tune a boundary weight.

4.6. Causal τ -curriculum

Forward operators that train simultaneously across all time-to-maturities can be pulled toward spurious solutions at long τ before the short- τ region has converged, because the residual at long τ depends causally on the short- τ solution [19]. We address this with a causal curriculum: training begins at $\tau_{\text{max}} = 0.15T$ and linearly ramps to the full T over the first half of training. The short- τ solution is therefore established before the long- τ residual can perturb it.

Algorithm 1 Training the PINN forward operator for the lifted rough-Heston PDE

Require: Model parameters $(V_0, \lambda, \theta, \nu, \rho)$; lift $(c_i, x_i)_{i=1}^n$; maturity T ; hyperparameters in Table 1

- 1: Initialise MLP N_θ (Xavier normal); learnable $s = [s_1, s_2, s_3] \leftarrow 0$
- 2: **Pre-flight MC:** simulate lifted SDE, record $(\hat{\mu}_i, \hat{\sigma}_i)$ for each factor and term structure $w(\tau)$ $\triangleright$ Sections 4.4, 4.3
- 3: **for** iteration $t = 1, \dots, N_{\text{iter}}$ **do**
- 4: $\tau_{\text{max}} \leftarrow \min(T, 0.15T + \frac{0.85T \cdot t}{0.5N_{\text{iter}}})$ $\triangleright$ τ -curriculum, Sec. 4.6
- 5: Sample interior: $\{(\tau_j, x_j, u_j)\}$ from $\text{Unif}([0, \tau_{\text{max}}]) \times \text{Unif}(x_{\text{lo}}, x_{\text{hi}}) \times \mathcal{N}(\hat{\mu}, \hat{\sigma})$
- 6: Sample boundary: $\{(\tau_j^{\text{bc}}, x_{\text{lo/hi}}, u_j^{\text{bc}})\}$
- 7: $\hat{P} \leftarrow P_{\text{base}}(\tau, x; w) + \tau N_\theta(\tau, x, u)$ $\triangleright$ Ansatz, Eq. (5)
- 8: $\mathcal{L}_{\text{PDE}} \leftarrow$ mean-squared PDE residual of $\hat{P}$ (Eq. (4))
- 9: $\mathcal{L}_{\text{OTM}} \leftarrow \|\hat{P}(\tau^{\text{bc}}, x_{\text{lo}}, u^{\text{bc}})\|^2$, $\mathcal{L}_{\text{ITM}} \leftarrow \|\hat{P}(\tau^{\text{bc}}, x_{\text{hi}}, u^{\text{bc}}) - (e^{x_{\text{hi}}} - Ke^{-r\tau^{\text{bc}}})\|^2$
- 10: $\mathcal{L} \leftarrow \sum_k [e^{-s_k} \mathcal{L}_k + s_k]$ $\triangleright$ Adaptive weighting, Eq. (7)
- 11: Backpropagate $\mathcal{L}$; clip gradients ($\ell_2 \leq 1.0$); Adam step on (N_θ, s) ; cosine LR anneal
- 12: **end for**

4.7. Training algorithm

Algorithm 1 summarises the full training procedure for the forward operator.

5. Evaluation methodology: trusting the forward operator

5.1. Two independent ground truths

An AI forward operator used inside an inverse problem is only as trustworthy as its own error against the true forward map. We hold the network to two independent references.

Fourier forward model. A semi-analytic pricer that solves the fractional Riccati equation for the rough Heston characteristic function [5] and inverts by Fourier integration [20]. It never uses the lift, so it is the ground truth of the *true, non-lifted model* ($n \rightarrow \infty$); in the flat-volatility limit it reproduces Black–Scholes to 3.7×10^{-7} in price.

Lifted Monte Carlo. An exponential-integrator Euler scheme that integrates the n -factor lifted SDE with the $-x_i U^i$ drift handled exactly:

$$U_{t+\Delta t}^i = e^{-x_i \Delta t} U_t^i + \int_t^{t+\Delta t} e^{-x_i(t+\Delta t-s)} \sqrt{V_s} dW_s^V. \quad (8)$$

The exponential-integrator is essential: the mean reversions (x_i) span roughly ten orders of magnitude for small H , and a naive explicit Euler scheme applied to the factor equations diverges. The lifted MC matches Black–Scholes within Monte Carlo error in the flat-variance degeneracy ($\nu=\lambda=0$; 100,000 paths, 200 steps) and agrees with the Fourier forward model for large n . At finite n , the lifted MC is the *exact solution of the forward PDE the network is asked to solve*; the Fourier forward model is the exact solution of the true model.

Precision of the reference. Since every solve-error in this paper is measured against this simulator, its own error budget has to be smaller than the quantity being measured, and we state it explicitly rather than leave it implicit. All validation runs use 10^6 antithetic paths and 1600 time steps with a terminal-forward control variate. The statistical standard error is 0.69 px bp at $n=4$ rising to 0.81 px bp at $n=32$, against solve-errors of 15–22 px bp. Two points deserve emphasis. First, the standard error must be computed on the antithetic *pair means*, not on the individual paths; the naive independent-sample formula is not a valid estimator here and overstates the RMS error by 14–31 per cent. Second, the time-discretisation bias is a systematic error that no amount of sampling removes, and it is the more dangerous of the two. A common-random-numbers refinement study (Table 2) measures it at 0.21, 0.17, 0.82 and 1.27 px bp for $n=4, 8, 16, 32$ at 1600 steps. We record why the step count is 1600 and not the 400 of the submitted version, because the choice was forced by the data. At the calibration of Section 7.4 the factor system is considerably stiffer than at the parameters we first validated on ($\nu = 1.37$ and $\lambda = 4.94$ against 0.30 and 2.57), and at 400 steps the bias grows to 3.2 and 3.8 px bp at $n=16$ and 32. That is comparable to the lift-error being measured, and it was visible: against the 400-step reference the lift-error at those n stopped falling and changed sign. At 1600 steps the bias is below a tenth of every solve-error in this paper.

5.2. Error decomposition

Rather than reporting a single “network-versus-true-model” figure, we decompose the error per strike and in price space (where the signed contri-

Table 2: Precision of the lifted Monte Carlo reference at the calibration of Section 7.4, $T=0.15$, in price basis points. Standard errors are RMS over strikes; the naive column is the independent-sample formula, which is not valid under antithetic sampling and is shown only to quantify the overstatement. Time-discretisation bias is from a common-random-numbers refinement against 3 200 steps.

n	SE, 2×10^5	naive SE	SE, 10^6	bias, 400	bias, 1 600
4	1.53	2.00	0.69	0.56	0.21
8	1.67	2.02	0.75	0.65	0.17
16	1.76	2.04	0.79	3.21	0.82
32	1.80	2.05	0.81	3.81	1.27

butions add exactly),

$$\underbrace{(\text{PINN} - \text{Fourier})}_{\text{total forward error}} = \underbrace{(\text{PINN} - \text{MC})}_{\text{network solve-error}} + \underbrace{(\text{MC} - \text{Fourier})}_{\text{lift (model) error}}. \quad (9)$$

The *solve-error* measures how accurately the network solves the forward PDE it was given; it is the quantity training controls, and it is the AI method’s own contribution to inverse-problem error. The *lift-error* is a forward-model approximation that only increasing n can reduce; it is not the network’s responsibility, and no amount of network re-training removes it. Conflating the two gives a systematically pessimistic view of the AI method and leads to the wrong engineering response inside a calibration pipeline—e.g. training longer when the correct response is to raise n .

Units, stated once. Two units appear in this paper and a referee rightly asked us to connect them explicitly. Price errors are in basis points of spot (px bp; $1 \text{ px bp} = 10^{-4} \times S$), and we use these wherever the decomposition (9) is invoked, because it is additive in price and *not* in implied volatility. Implied-volatility errors are in vol bp ($1 \text{ vol bp} = 10^{-4}$ in volatility), which is the unit a practitioner reads. The two are related by the Black–Scholes vega: at the money, $T=0.15$ and $S=1$, vega is 0.154 almost independently of the volatility level, so

$$100 \text{ vol bp} \approx 15.4 \text{ px bp} \quad (\text{at the money, } T=0.15).$$

The conversion is pointwise, not global: vega falls away in the wings, so an RMSE aggregated over the strike range 0.85–1.15 converts at roughly 12–14

px bp per 100 vol bp rather than 15.4. This is why, for instance, a lift-error of 16.4 px bp in Table 4 corresponds to 133 vol bp in Section 7.3 rather than to 106. To keep the two readable side by side, **Table 4 now reports both units for every entry**. Finally, note that the two components of (9) carry opposite signs, so their RMSE values do not add even though the signed errors do.

5.3. Homogeneity re-scaling

We train the network at fixed strike $K=1$ and read off the smile $C(1, K)$ through the call’s degree-one homogeneity:

$$C(1, K) = K C(1/K, 1), \quad (10)$$

which holds exactly because the variance dynamics are independent of the spot level. Dropping this re-scaling and comparing mismatched options inflates the apparent error to ~ 2700 vol bp; we verified this is a comparison artefact, not a modelling failure of the forward operator.

6. The calibration inverse problem: data and methods

6.1. Data

We use a single BTC option-chain snapshot from Deribit, a cryptocurrency derivatives exchange, retrieved through its public API (no API key required) on **3 July 2026 at 05:57 UTC**, at a spot of \$62,191. The raw snapshot holds 912 quoted instruments across 13 expiries.

Because the referees asked for the data handling to be stated precisely, we give the filter chain in full. From the raw chain we (i) take the mid price $\frac{1}{2}(\text{bid} + \text{ask})$ where a two-sided quote exists and otherwise fall back to Deribit’s *mark* price; (ii) keep only the out-of-the-money leg at each strike (calls for $K \geq F$, puts for $K < F$), which is the more liquid and tighter-quoted side; (iii) drop $|k| > 0.30$ with $k = \log(K/F)$; (iv) drop quotes below one currency unit, where the implied vol is dominated by tick noise; and (v) recompute the implied volatility from the mid by Brent inversion, dropping any point falling outside $[0.01, 2.0]$. This leaves 286 points, from which we retain at most eight per maturity, spread evenly in log-moneyness so that each smile constrains shape and not merely level. The fitted surface therefore comprises **104 points across 13 maturities** (T from 0.002 to 0.98 years) spanning a moneyness range $K/F \in [0.77, 1.35]$; of these, 97 return a finite

model implied volatility at the calibrated parameters and enter the reported RMSE.

Two disclosures about that surface. First, **11 of the 104 fitted points carry no two-sided quote** and therefore enter through Deribit’s mark price, which is an exchange model value rather than a tradable one. Second, **12 of the 104 had no traded volume** in the snapshot window. We do not filter on volume or open interest: Deribit’s book-summary endpoint does not return open interest, and imposing a volume filter would remove most of the long maturities entirely. Section 7.4 reports the fit restricted to the liquid moneyness core as a robustness check, and Section 6.2 reports static-arbitrage diagnostics on the surface that survives the filters above.

Two further datasets, used only for robustness. The regime and roughness checks of Section 7.5 use two datasets distinct from the headline snapshot, and we describe both here so that their caveats sit with the data rather than with the results. (i) A five-minute bar history of the Deribit BTC perpetual, August 2018 to August 2026 (841k bars), collected by the authors through the same public API. It enters only the realised-volatility estimate of H and never touches an option. (ii) A daily implied-volatility panel of BTC options, December 2016 to August 2026, built by the authors from the Deribit public trade tape (41.5M option trades). Every entry in that panel is a completed transaction, not a quote: there is no bid–ask, the two-sided-quote filter above cannot be applied, and a single stale print can set the implied volatility of an illiquid strike. We therefore restrict the panel to $|k| \leq 0.35$, $T \in [0.02, 1.0]$ years and implied volatilities in $[0.01, 2.0]$, retain at most eight points per maturity spread in log-moneyness exactly as above, and keep only Wednesdays with at least four maturities and 25 points. Trade-based and quote-based surfaces are each internally consistent but are not interchangeable: a comparison across weeks of the panel is sound, but the panel is not a like-for-like extension of the headline snapshot, and we do not present it as one. Both datasets accompany the code in derived form (the weekly surfaces and the daily realised variance), not as raw tapes.

We set $r=0$, the standard convention for crypto, where there is no clean carry, and take the forward to be $F = S e^{rT} = S$ with S the underlying index price Deribit reports alongside each instrument. Implied volatilities are computed from mid prices with a robust Brent Black–Scholes inverter. The inverse-problem objective throughout is the vega-weighted implied-volatility

RMSE between market quotes and the forward operator’s output,

$$\mathcal{L}(\Theta) = \sqrt{\frac{\sum_j w_j [\sigma_j^{\text{mod}}(\Theta) - \sigma_j^{\text{mkt}}]^2}{\sum_j w_j}}, \quad w_j = \max\left(S \phi(d_{1,j}) \sqrt{T_j}, 10^{-6}\right), \quad (11)$$

with ϕ the standard normal density and $d_{1,j}$ evaluated at the market implied volatility, so the weights are fixed across the optimisation. Vega weighting lets well-determined near-the-money points dominate, which is standard practice. We flag one asymmetry for the reader: the *objective* is vega-weighted as in (11), whereas the headline fit quality we report in Section 7.4 is the *un-weighted* RMSE over the same points, which is the more conservative of the two.

6.2. Static-arbitrage diagnostics

Attributing a coarse fit to the forward model’s expressiveness, as we do in Section 7.4, is only legitimate if the target surface is itself arbitrage-consistent: quotes carrying butterfly or calendar violations cannot be fitted by *any* arbitrage-free model, and the residual would then be the data’s doing rather than rough Heston’s. Crypto surfaces are wide and thinly quoted in the wings, so this is a real possibility and we test it rather than assume it.

On undiscounted calls with $r=0$, putting the out-of-the-money puts on a common footing through parity, we check three conditions on the filtered surface: convexity in strike at each maturity (butterfly), the slope bound $-1 \leq \partial C / \partial K \leq 0$ (vertical spread), and monotonicity of total implied variance $w(k, T) = \sigma^2 T$ in T at fixed log-moneyness (calendar). The result is clean: **no violations** in 78 butterfly triples, 91 vertical pairs or 300 calendar comparisons. The margins are however slim — the smallest butterfly margin is 0.28 px bp and the smallest calendar increment is 8.2×10^{-5} in total-variance units — so the surface is arbitrage-free but only barely, which is itself a fair description of a wide crypto smile. The calibrated model’s own surface passes the same tests with larger margins (0.98 px bp butterfly), as an arbitrage-free model must.

Separately, put–call parity holds on the raw chain: of 383 strikes quoted two-sided on both legs, **none** violates $C - P = F - K$ by more than the quoted bid–ask spread, though the residual RMS of 46 px bp (maximum 163) is a reminder of how wide those spreads are.

The conclusion we draw is narrow but useful: the coarseness of the fit reported in Section 7.4 *cannot* be blamed on static-arbitrage violations in the target, because there are none. It is a statement about the forward model.

The liquid core. The sharper form of the referee’s question is whether the coarse fit is driven by the wings. Table 3 refits on the liquid core, $|k| \leq 0.10$ and $|k| \leq 0.20$, from both ends of the H ridge of Section 7.5, and also evaluates the two unrefitted full-surface vectors on the same core points, which separates the effect of dropping the wings from the effect of re-optimising. Two things follow from Table 3, and the comparison that matters is on a common set of points. First, the wings are not what makes the fit coarse. On the fifty-three points with $|k| \leq 0.10$, the full-surface fit scores 160.4 vol bp weighted and 510.8 unweighted; refitting on those points alone improves this only to 146.4 and 450.0, a gain of nine to twelve per cent. A model that remains above 450 vol bp on the most liquid quotes it is shown is not being defeated by illiquid ones; the residual belongs to the lifted rough-Heston form. Second, and more interesting, the recovered H depends strongly on the window: 0.020 on $|k| \leq 0.10$, 0.057 on $|k| \leq 0.20$, and 0.172 on the full surface. On both restricted windows the two warm starts converge to the same parameters—to 0.2 vol bp on the core and to the reported precision on $|k| \leq 0.20$ —so there H is identified. The non-identification of Section 7.5 is therefore specifically a property of the far wings, which is where the smoother-kernel, higher-vol-of-vol end of the ridge buys its fit.

Table 3: Liquid-core refit. Fit quality in vol bp, weighted by the vega weights of the objective and unweighted. Unrefitted rows evaluate the full-surface vectors on the restricted points.

Window	pts	Fit	weighted	unweighted	H	ρ	ν
full	104	full-surface fit	113.2	416.4	—	—	—
$ k \leq 0.10$	53	full-surface fit	160.4	510.8	—	—	—
		refit, start $H=0.17$	146.4	450.0	0.020	−0.360	0.628
		refit, start $H=0.25$	146.6	460.1	0.022	−0.369	0.647
$ k \leq 0.20$	83	full-surface fit	127.0	461.7	—	—	—
		refit, start $H=0.17$	125.0	468.1	0.057	−0.410	0.826
		refit, start $H=0.25$	125.0	467.6	0.057	−0.410	0.826

6.3. Three routes to the inverse problem

We compare three ways of solving the same calibration inverse problem.

1. **True-forward-model loop.** Differential evolution (60 generations, all 5 parameters) with the Fourier forward model inside the objective, followed by Nelder–Mead polish. This is our from-scratch, exact-forward-operator reference.
2. **AI-only.** Gradient descent on the model parameters using the parametric PINN as a differentiable forward operator, with zero calls to the true forward model.
3. **Hybrid.** The parametric PINN’s single-shot solution provides the initial guess; a short Nelder–Mead polish against the true forward model then sharpens it.

Optimiser settings, stated once. Two distinct budgets appear in this paper and conflating them would misstate the speed-up, so we separate them explicitly. The *full-budget market calibration* of Section 7.4 uses SciPy’s `differential_evolution` with `maxiter=60`, `popsiz=12`, `tol=10-4`, mutation (0.5, 1.0), recombination 0.7 and seed 1, the built-in polish disabled, evaluated in parallel with deferred population updating, followed by a separate Nelder–Mead refinement (`maxiter=200`, `xatol=10-4`, `fatol=10-6`) confined to the box. The box is $H \in [0.02, 0.45]$, $V_0, \theta \in [0.005, 0.60]$, $\lambda \in [0.05, 12]$, $\nu \in [0.05, 2]$, $\rho \in [-0.95, -0.05]$; every parameter of the reported fit is interior to it. The *three-route comparison* of this section runs on the single $T=0.15$ slice with a deliberately lighter global budget—`maxiter=30`, `popsiz=10`, seed 1—and a Nelder–Mead polish capped at 120 iterations. All forward-model evaluation counts we report, including the 1 736-evaluation from-scratch baseline against which the $8.8\times$ saving is measured, come from this lighter configuration; they are not comparable to the full-budget market fit, and we do not claim they are.

We also record how the box was arrived at, because it was tight three times over and each time it moved the answer. In the submitted fit the Nelder–Mead refinement ran *unbounded* and θ escaped the interval the global search had explored. Under the corrected objective, a box capping λ at 3 left λ on its bound; the box above is wide enough to contain every parameter. In the profile likelihood of Section 7.5 the same cap held λ on its bound at ten of twelve grid points and narrowed the reported H interval by almost half until it was lifted. A search box that constrains rather than contains the optimum is a quiet source of false precision, and we now check every reported fit for bound contacts.

7. Results

7.1. *The forward operator scales where the grid cannot*

Our headline forward-operator experiment trains one architecture (width 64, depth 4) at $n=4, 8, 16, 32$ on the BTC calibration of Section 7.4 at $T=0.15$, over eight independent seeds per n (Table 4, Figure 1). The solve-error against the lifted MC is 15.2 ± 4.1 , 21.2 ± 5.2 , 24.6 ± 5.0 and 18.3 ± 6.1 px bp at $n=4, 8, 16, 32$. It rises to $n=16$ and falls back at $n=32$, and across the eight-fold rise in dimension it grows by a factor of 1.21. We test the question the scaling claim turns on directly, on all thirty-two seed-level points. A least-squares fit of log solve-error on log n gives a slope of 0.09 ± 0.08 , significantly below one, so the growth is sub-linear; and a one-sided permutation test of monotonically increasing error, over 20 000 relabellings, gives $p = 0.14$, so we cannot reject that the solve-error is flat in n . The submitted version reported a doubling on three seeds whose error bars overlapped at adjacent n ; with eight seeds the defensible statement is the stronger one, that we detect no growth at all. The absolute level is higher than in the submitted version because every experiment now runs at the corrected calibration, whose fast mean-reversion ($\lambda = 4.94$) and large vol-of-vol ($\nu = 1.37$) make the lifted factor system markedly stiffer. Training cost does not grow with n either: measured on an otherwise idle machine, one training iteration costs 84–120 ms at every n , with no trend beyond the scatter between repeats, because the factor-Hessian computation is vectorised over the full double sum.

The lift-error falls from 8.8 px bp at $n=4$ to 3.8 at $n=8$ and reaches the resolution floor of the reference by $n=16$ (shaded in Figure 1): the Monte Carlo reference’s standard error and time-discretisation bias together amount to 1.8 px bp at $n=16$ and 2.7 at $n=32$, and the lift-errors measured there, 1.0 and 3.4, are not resolvable against it. Two checks show the floor is the reference and not the Fourier price: the Fourier price is converged to 0.1 vol bp across a sixteen-fold refinement of both its time grid and its frequency truncation, and the signed lift-error rises monotonically in n at every strike, crossing zero between $n=8$ and 16 at and above the money and between 16 and 32 in the put wing. That is the signature of a kernel-approximation error that vanishes with n meeting a positive discretisation bias that grows with it, which is what the floor itself shows. We report the lift-error beyond $n=16$ as below the floor and draw no conclusion from its value there.

Compare this against a finite-difference forward solver. A naive full-tensor grid on the same $(n+1)$ -dimensional PDE at a modest 50 nodes per dimension

needs 50^{n+1} points: 3.1×10^8 at $n=4$, 2.0×10^{15} at $n=8$, and 1.2×10^{56} at $n=32$. Even $n=8$ —roughly 16 petabytes in double precision—is out of reach as a forward operator inside any iterative inverse-problem solver, while the network trains every n on a laptop. We stress that 50^{n+1} is an illustrative memory bound for the naive full-tensor discretisation and not a benchmark against a tuned solver. Structured schemes reach considerably further: sparse grids [23, 24] break the full-tensor node count, ADI splitting handles the correlation cross-terms efficiently in the classical Heston setting [25], and low-rank tensor-train representations [26] compress high-dimensional operators where the solution admits low-rank structure. Each, however, still degrades with n —sparse-grid node counts and error constants grow super-linearly in the dimension, and the tensor-train ranks that make the method cheap are eroded by precisely the coupled ρ - ν cross-terms this PDE carries—and none is standard or validated for the lifted rough-volatility PDE. We therefore make the conservative claim, and only it: a competitive grid-based forward solver at $n=16$ – 32 is *impractical* as the engine of a calibration loop, not provably impossible. Establishing the stronger claim would require building and tuning such a solver, which is a research programme in its own right.

Table 4: Dimension scaling of the forward operator: single-point PINN at $T=0.15$ on the BTC calibration of Section 7.4. Solve-error is the eight-seed mean $\pm$ SD in price basis points of spot, against a lifted Monte Carlo of 10^6 antithetic paths at 1 600 steps; lift-errors at $n=16$ and 32 are below that reference’s resolution floor (see text). Cost is milliseconds per training iteration, measured with no other job running.

n	solve-error (px bp)	lift-error (px bp)	ms / iteration	FD grid $50^{(n+1)}$
4	15.2 ± 4.1	8.8	120	3.1×10^8
8	21.2 ± 5.2	3.8	94	2.0×10^{15}
16	24.6 ± 5.0	1.0	92	7.6×10^{28}
32	18.3 ± 6.1	3.4	84	1.2×10^{56}

7.2. Sensitivity to architecture and collocation, and where the variation lives

Two questions follow from Table 4. Is the width-64 / depth-4 choice a knife-edge? And the solve-error, though it shows no significant trend in n , does rise and fall across n by up to sixty per cent: does that variation come from the lifted PDE or from the optimiser? Table 5 answers the first with a 3×3 architecture grid at $n=4$. For the second we use the a-priori structure of PINN error estimates [27, 28]: the solve-error is bounded by a stability

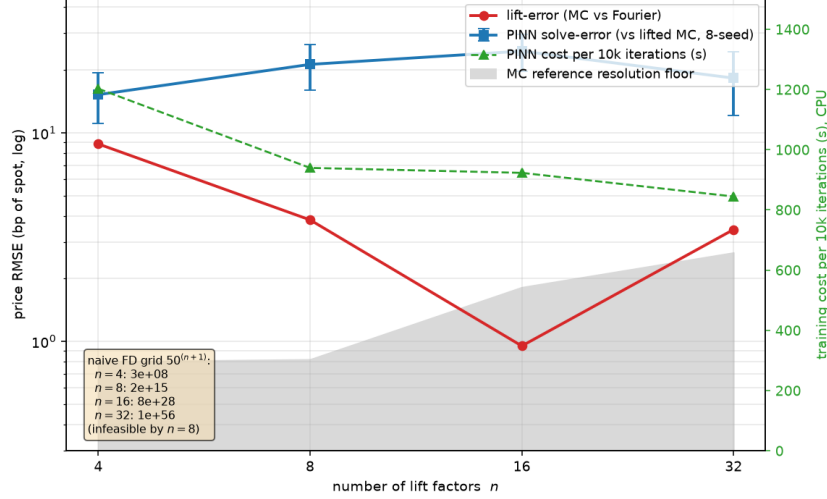


Figure 1: Dimension scaling of the forward operator. PINN solve-error (blue, eight-seed mean $\pm$ SD) shows no statistically detectable growth over an eight-fold rise in n (log-log slope 0.09 ± 0.08 ; permutation $p = 0.14$), and training cost per iteration (green) is flat. The lift-error (red) falls into the Monte Carlo reference’s resolution floor (grey: standard error combined with time-discretisation bias) by $n=16$, beyond which it is not resolvable. The inset shows the naive finite-difference grid size 50^{n+1} , infeasible by $n=8$.

constant of the PDE times the residual, so we score every trained network on a fixed, seeded, never-trained-on set of 20 000 collocation points and follow that held-out residual across n (Table 6). If the residual is flat while the solve-error moves, the variation sits in the stability constant $C(n)$; if the residual moves with it, the cause is optimisation or quadrature, and Table 7 tests the quadrature half directly by raising the collocation count at $n=32$, where the paper holds n_{col} at 2000 while the dimension is 33. Three findings, the first two against us. The architecture used throughout this paper is not the best one available: width 128 at depth 4 reaches 9.7 ± 0.5 px bp at $n=4$ against 18.2 ± 4.2 for our width 64, with four times as many parameters, and we report the headline results at width 64 only for continuity with the submitted version. Second, collocation is under-resolved at high dimension: at $n=32$ raising n_{col} from 2000 to 4000 halves the solve-error (22.1 to 11.0 px bp), which is the quadrature term of the error estimate at work. Third, and this is the answer to where the variation across n comes from, the held-out residual is flat across n to within twelve per cent while the solve-error moves by up to fifty-seven per cent, with the same rise to $n=16$ and fall at

$n=32$ as the eight-seed sweep of Table 4. The network solves the PDE equally well at every dimension, so whatever variation there is in solve-error lies in the factor that converts residual into solution error, the stability constant $C(n)$ of the lifted operator [27, 28]—a property of the Markovian lift, not of the optimiser. With two seeds per cell here and no significant trend in the eight-seed sweep, we offer this as where the variation lives, not as evidence that the error grows with n . The minutes-per-run column of Table 5 was recorded while other jobs shared the machine and is indicative only; Table 4 gives the controlled cost.

Table 5: Architecture grid at $n=4$, two seeds, 10^4 iterations. Solve-error in px bp against the 1M-path lifted MC; held-out PDE residual (RMS); minutes per run on CPU.

width	depth	params	solve-error	residual	min
32	3	2369	23.0 ± 2.1	$3.77e - 02$	15.3
32	4	3425	20.9 ± 8.0	$2.73e - 02$	20.5
32	5	4481	24.2 ± 0.3	$2.68e - 02$	21.6
64	3	8833	20.0 ± 2.6	$2.52e - 02$	20.4
64	4	12993	18.2 ± 4.2	$2.08e - 02$	25.5
64	5	17153	24.2 ± 2.3	$1.65e - 02$	27.5
128	3	34049	11.9 ± 3.2	$1.98e - 02$	33.2
128	4	50561	9.7 ± 0.4	$1.36e - 02$	33.1
128	5	67073	16.1 ± 2.3	$1.03e - 02$	28.2

Table 6: Held-out PDE residual across n (width 64, depth 4, $n_{\text{col}}=2000$, two seeds), with the solve-error and both quantities relative to $n=4$.

n	solve-error	residual	solve/solve ₄	resid./resid. ₄
4	18.2 ± 4.2	$2.08e - 02$	1.00	1.00
8	26.7 ± 7.5	$1.93e - 02$	1.47	0.93
16	28.5 ± 3.1	$2.16e - 02$	1.57	1.04
32	22.1 ± 10.4	$2.00e - 02$	1.22	0.96

7.3. Error decomposition and a design rule for n

The value of the decomposition (9) is that it tells a user which lever to pull, and the answer depends on the calibration. We have seen it both ways in this work. At the parameters of the submitted version ($H=0.09$, $\nu=0.30$),

Table 7: Collocation ablation at $n=32$ (width 64, depth 4, two seeds).

n_{col}	solve-error	residual	min
2000	22.1 ± 10.4	$2.00e - 02$	8.7
4000	11.0 ± 7.7	$1.52e - 02$	14.0
8000	11.7 ± 0.6	$1.29e - 02$	35.9

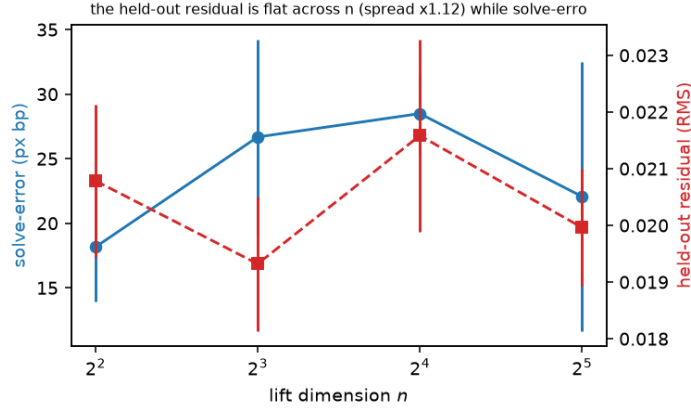


Figure 2: Solve-error (left axis) and held-out PDE residual (right axis) across the lift dimension n , width 64 / depth 4, two seeds.

a single “network versus true model” error of 121 vol bp at $n=4$ looked like a failure of the network; split by (9) it was almost entirely lift—133 vol bp of lift-error against 37 of solve-error—and the lever was more factors. At the corrected calibration the split reverses (implied-vol RMS, seed-averaged):

n	solve-error	lift-error	total (vol bp)
4	112	76	80
8	152	33	130
16	176	9	177
32	136	26	158

The solve-error exceeds the lift-error at every n , so here the network is the binding constraint and more factors buy nothing: the lever is the network itself, and Section 7.2 shows it is a real one, since a wider network or twice the collocation roughly halves the solve-error. The two error components also partly cancel at small n , which is why the total at $n=4$ is smaller than

the solve-error alone; a single end-to-end figure would have hidden both the sign and the source of the error.

This gives a design rule that holds in both regimes: *add lift factors only while the lift-error exceeds the solve-error; beyond that, improve the network.* The crossover is calibration-dependent. At the submitted parameters it fell near $n=8-10$; at the corrected fit, whose smoother kernel ($H=0.17$) is easier to approximate and whose stiffer factor dynamics are harder to solve, the lift-error is below the solve-error already at $n=4$. A robustness sweep run before this revision, at the submitted parameters and against a 400-step reference, found the crossover shifting with maturity in the expected direction: the lift-error at $n=4$ grew from 8.6 to 31.8 px bp as T rose from 0.05 to 0.5, so longer-maturity calibration needs more factors.

The network’s error is not symmetric in strike. At every n it over-prices at and above the money, by +138 to +221 vol bp ATM and +130 to +203 in the calls, while the put wing is smaller and at $n=32$ near zero (−15). This is a watch item for anyone deploying the operator inside a calibration loop, since a one-signed error biases the recovered skew; we examine its mechanism next.

The $n=32$ bias, taken apart. Factor-tail coverage is symmetric by construction: the sampler widens both signs of the factor deviation, and the asymmetric option is off by default, so asymmetry is not the mechanism. Two others are separable by ablation: a noisy forward-variance anchor (the baseline is anchored on a pre-flight Monte Carlo of 4 000 paths, and a level error in the anchor is a level error in the price, which is what an ATM/call bias looks like), and under-coverage of the factor tails in 32 dimensions. Table 8 re-estimates the anchor with 200 000 paths and widens the tails, separately and together, against the same reference of 10^6 antithetic paths at 1 600 time steps, whose own discretisation bias at this n is 1.3 px bp (Table 2). The ablation separates the two cleanly. A precise anchor lowers the solve-error from 16.8 to 14.3 px bp and the ATM bias by thirty vol bp, so a noisy anchor is part of the mechanism. Widening the tails does the opposite: it raises the solve-error by 86 per cent, roughly doubles the ATM bias and raises the call bias by seventy per cent, alone or combined with the precise anchor. Wider tails spend collocation points in regions of factor space the solution rarely visits, at the expense of the bulk where the price is determined. The residual bias that survives a precise anchor is the growth with n that Section 7.2 attributes to the stability constant of the lifted PDE. All figures in this table

are scored against a 1 600-step reference; at the 400 steps originally used, the reference itself priced high by 3–4 px bp at this n and masked part of the network’s over-pricing.

Table 8: Ablation of the $n=32$ directional bias, three seeds. Solve-error in px bp; signed implied-vol error by strike bucket in vol bp (network minus lifted MC).

Configuration	solve-error	puts	ATM	calls
baseline (J1 setting)	16.8 ± 10.1	+3	+132	+186
precise anchor	14.3 ± 5.8	−17	+102	+162
wide tails	31.3 ± 8.2	+124	+261	+315
precise anchor + wide tails	32.3 ± 9.4	+84	+268	+338
Reference: 10^6 paths, 1 600 steps; its own bias 1.27 px bp.				

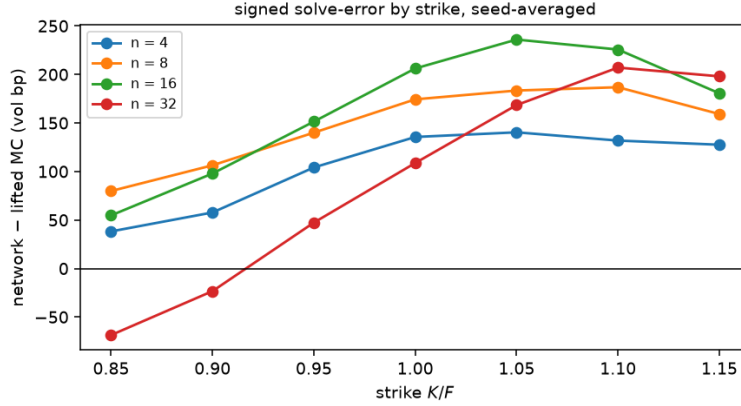


Figure 3: Signed solve-error by strike (network minus lifted MC, implied-vol basis points), seed-averaged, at $n=4, 8, 16, 32$. The put wing stays near zero while the ATM and call errors grow with n .

7.4. Solving the inverse problem: the correlation is pinned, the roughness is not

Calibrated to the full Deribit BTC surface with a full-budget optimiser (differential evolution, 60 generations, all 13 maturities and all 104 points, followed by a bounded Nelder–Mead polish against the true Fourier forward model), rough Heston returns

$$H=0.172, \quad \rho=-0.426, \quad V_0=0.105, \quad \theta=0.245, \quad \lambda=4.94, \quad \nu=1.37,$$

at a vega-weighted implied-vol RMSE of 113 vol bp and an unweighted RMSE of 416 vol bp (Figure 4). The unweighted figure is dominated by one part of the surface. On the thirty-two points with maturities beyond two months the model fits to 52 vol bp, comfortably inside a quoted Bitcoin option spread; on the forty-eight points under two weeks it misses by 608. Rough Heston describes the BTC term structure well and fails only on the shortest-dated options, whose smiles are too steep for this model to follow. That is a statement about the forward model’s expressiveness, not about the AI forward operator, and it is orthogonal to the solve-error studied above.

A correction to the submitted fit. The submitted version reported $H=0.090$, $\rho=-0.792$, $\nu=0.300$ at 496 vol bp. That fit was produced by an objective with a defect we found during this revision, and we state it plainly because it changes the headline parameters. When the Fourier price at a point fell at or below intrinsic value, the implied-volatility inversion returned no value and the objective silently dropped the point and renormalised over the rest. A parameter vector at which badly fitted points *failed to price* therefore scored better than one that priced them, and the optimiser could lower the loss by breaking the pricer. The submitted $H=0.090$ sits in a window of width 2×10^{-4} in H created in exactly that way. A second defect compounded it: the Lewis integrand decays like $\exp(-VTu^2/2)$, so its truncation frequency must scale with $1/\sqrt{VT}$, and a fixed truncation mis-priced the shortest maturities by up to 0.33 in implied volatility. Both are corrected. Every point now counts in every evaluation, a point priced at or below intrinsic enters at the floor of the inversion bracket so that its full error is charged, and a genuine pricer failure makes the whole parameter vector infeasible rather than lighter. Scored under the corrected objective the submitted vector reaches 204 vol bp weighted against 113 for the fit above.

The correlation and the vol-of-vol moved substantially; the roughness did not move in the way that matters. Section 7.5 shows that this surface pins ρ to within a single grid point but leaves H free over $[0.04, 0.30]$ at five per cent of the best fit, so the point value 0.172 above is one draw from that interval and not a measurement.

7.5. *Is the recovered roughness real, and is it stable?*

Whether Bitcoin volatility is rough rests, so far, on one option surface read through a six-parameter inverse problem. Two things could undermine the claim. The inverse problem might not identify H sharply, in which case

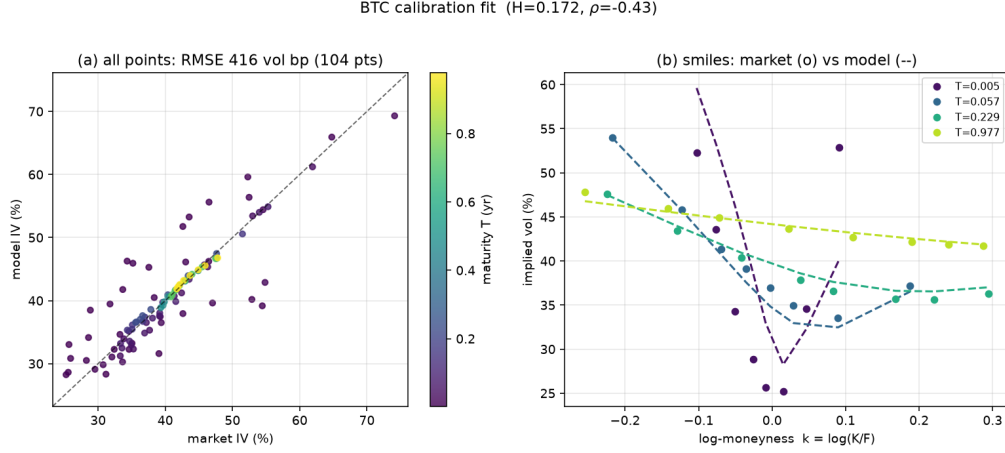


Figure 4: Rough Heston calibrated to the Deribit BTC surface (all 104 points, 13 maturities), solving the inverse problem with the true Fourier forward model. (a) Market versus model implied volatility at every point (dashed: identity; colour: maturity); the outliers are the shortest maturities. (b) Per-maturity smiles for four representative maturities: market (markers) versus model (dashed). Beyond two months the model fits to 52 vol bp; the residual is concentrated in options under two weeks and reflects a model-expressiveness limitation, not a forward-operator artefact.

any single fitted value would be one point on a flat ridge rather than a measurement; and even if it does, a single snapshot says nothing about whether the roughness is a stable feature of the market or an artefact of one day's regime. We test both, the second using an estimator that never touches an option price.

Whether the surface identifies H . It does not. The first evidence is simply that three full-budget fits of the same surface under the same corrected objective, differing only in the search box, disagree about H by a factor of nine while agreeing about everything a reader would check:

search box	H	ν	ρ	weighted	unweighted
original, unbounded polish	0.019	0.75	-0.419	114.9	453.7
widened	0.096	0.98	-0.421	113.7	430.2
widened further	0.172	1.37	-0.426	113.2	416.4

The weighted objective differs by 1.7 vol bp across the three, and ρ by 0.007. The loss surface has a long, nearly flat ridge in H along which ν compensates:

a rougher kernel with a smaller vol-of-vol prices this surface about as well as a smoother kernel with a larger one. A profile likelihood makes this precise (Table 9, Figure 5). Fixing H on a twelve-point grid and re-optimising the other five parameters from both ends of the ridge, the weighted objective moves by only 1.3 volbp as H runs from 0.04 to 0.26, and every value of H in $[0.04, 0.30]$ fits within five per cent of the best. The mechanism is visible in the re-optimised parameters: ν climbs from 0.80 to 1.94 and λ from 2.3 to 7.6 along the ridge while ρ barely moves. Profiling ρ in the same way gives the opposite picture: the objective rises steeply on either side of $\rho = -0.45$ (146.5 volbp at -0.60 , 138.4 at -0.30 , against 115.0 at -0.45), so the five-per-cent set is a single grid point. *The surface identifies the correlation sharply and the roughness barely at all.* Because the whole H interval lies far below $\frac{1}{2}$, the qualitative claim that Bitcoin volatility is rough survives; the point estimate does not, and we do not report one. The profile’s minimum, 113.2 volbp at $H = 0.18$, matches the global fit; because each point is a local re-optimisation, the reported values are upper bounds and the interval is, if anything, slightly too narrow. An earlier pass of this profile confined λ to the box $[0.05, 3]$, where it sat on the upper bound at ten of twelve grid points; the figures above use a box wide enough to contain it, which widened the H interval from $[0.04, 0.18]$ to $[0.04, 0.30]$. One bound still binds, and it binds at the edge that matters: from $H=0.30$ upward the re-optimised ν sits on its cap of 2, so the profile there is itself box-limited and the upper end of the interval is a lower bound on the true one. The surface may support still smoother kernels than we report. We therefore do three things. We report the option-implied H as an interval rather than a point. We run every PINN experiment in this paper at the best fit, $H = 0.172$ with $\nu = 1.37$ and $\lambda = 4.94$, and we note that this is a deliberately demanding choice: the fast mean-reversion and large vol-of-vol make the lifted factor system markedly stiffer than at the parameters of the submitted version, and every solve-error in Section 7.1 is larger for it. And we lean on the estimate below, which never passes through this inverse problem at all.

What the ridge costs in risk terms. Non-identification of H matters only if the quantities a desk actually uses depend on where along the ridge one sits. Table 10 prices three of them with the exact forward model at three months, for every parameter vector the surface cannot tell apart: the variance-swap fair strike (from the log-contract replication), the 25-delta risk reversal, and a 90/110 collar. The answer is reassuring, and it follows from the mechanism

Table 9: Profile likelihood in H on the headline surface. H is fixed at each grid value and the other five parameters are re-optimised by Nelder–Mead from both ends of the ridge, keeping the better. Weighted and unweighted implied-vol RMSE in vol bp; the last three columns show how ν , ρ and λ compensate along the ridge.

H	weighted	unweighted	ν	ρ	λ
0.040	114.5	446.7	0.798	-0.421	2.298
0.060	114.1	440.4	0.869	-0.421	2.627
0.090	113.7	432.4	0.984	-0.423	3.155
0.120	113.4	425.5	1.111	-0.424	3.743
0.150	113.3	419.7	1.254	-0.425	4.406
0.180	113.2	415.3	1.414	-0.426	5.154
0.220	113.4	411.9	1.655	-0.428	6.290
0.260	113.8	411.4	1.935	-0.430	7.613
0.300	115.3	430.1	2.000	-0.435	7.720
0.350	120.8	475.9	2.000	-0.444	7.448
0.400	129.6	529.5	2.000	-0.454	7.306
0.450	140.8	582.4	2.000	-0.470	7.378

Best $H = 0.172$; 5% interval $[0.040, 0.300]$.

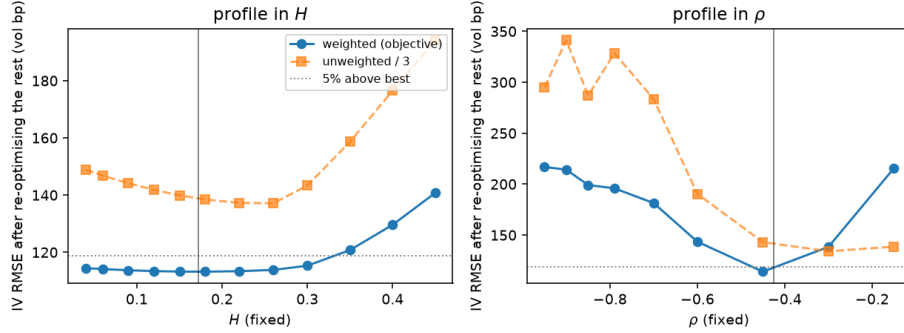


Figure 5: Profile likelihood on the headline surface. Left: H fixed on a grid, the other five parameters re-optimised from both ends of the ridge; right: the same for ρ . Vertical line: the canonical fit; dotted line: 5% above the best profile value, the interval rule.

above. Across the nine ridge vectors— H from 0.04 to 0.30, all within five per cent in fit—the variance-swap fair strike moves by 0.60 vol points on a level of 43.4, and the risk reversal by 0.46; the collar, which is sensitive to the curvature of the smile between its two strikes, moves by 5.7 px bp. A seven-fold ambiguity in H is therefore close to harmless for these functionals, because ν and λ compensate and the functionals depend on the compensated combination rather than on H alone. The practical corollary is the one a desk needs: what this surface cannot tell apart, these instruments do not care about. The three hybrid vectors of Section 7.6 tell a different story, and a useful one. They are fitted to the $T=0.15$ slice alone, and they price a risk reversal about half a vol point steeper and a collar seven basis points dearer than any full-surface fit: the ambiguity that matters in practice is not where on the ridge a calibration lands, but which part of the surface it was asked to fit.

Table 10: Risk functionals along the calibration ridge, Fourier-priced at $T=0.25$, $S_0=1$, $r=0$. Variance swap as $\sqrt{K_{\text{var}}}$ in annualised vol points; risk reversal as $\text{IV}(25\Delta \text{ call}) - \text{IV}(25\Delta \text{ put})$ in vol points; collar as long 90% put, short 110% call, in price basis points of spot.

Vector		Var. swap	25 Δ RR	Collar
reference fit	$H=0.172$	43.47	−6.63	+23.0
profile ridge	$H=0.040$ (+1.1%)	43.13	−6.42	+20.4
profile ridge	$H=0.060$ (+0.8%)	43.17	−6.45	+20.7
profile ridge	$H=0.090$ (+0.4%)	43.25	−6.49	+21.3
profile ridge	$H=0.120$ (+0.2%)	43.32	−6.54	+21.9
profile ridge	$H=0.150$ (+0.0%)	43.40	−6.59	+22.5
profile ridge	$H=0.180$ (+0.0%)	43.49	−6.65	+23.1
profile ridge	$H=0.220$ (+0.2%)	43.61	−6.72	+24.0
profile ridge	$H=0.260$ (+0.5%)	43.73	−6.80	+24.8
profile ridge	$H=0.300$ (+1.9%)	43.69	−6.88	+26.1
hybrid, seed 0	$H=0.071$	43.48	−7.17	+29.9
hybrid, seed 1	$H=0.120$	43.52	−7.19	+30.2
hybrid, seed 2	$H=0.078$	43.42	−7.15	+29.8
range, ridge (9)		0.60	0.46	5.7

An estimate that does not use the option surface. Following Gatheral, Jaisson and Rosenbaum [1], we estimate H directly from realised volatility. From

five-minute log returns of the Deribit BTC perpetual we build daily realised variance, set $\sigma_t = \sqrt{RV_t}$, and form the q -th absolute moment of log-volatility increments at lag Δ , $m(q, \Delta) = \mathbb{E}|\log \sigma_{t+\Delta} - \log \sigma_t|^q$. For a fractional process $m(q, \Delta) \sim \Delta^{qH}$, so regressing $\log m$ on $\log \Delta$ for each q gives slopes $\zeta(q)$, and regressing ζ on q through the origin gives H . Over 2 921 days (August 2018 to August 2026, 841k bars) this returns

$$H_{\mathbb{P}} = 0.111, \quad 95\% \text{ CI } [0.102, 0.150],$$

with the interval from a 60-day moving-block bootstrap. This is an independent corroboration in the strongest available sense: it uses no option data, no Markovian lift, no Fourier pricer and no optimiser, and it lands inside the option-implied interval $[0.04, 0.30]$. Where the option surface cannot choose a point on its ridge, this estimate can, and it chooses one well inside the rough regime. It also agrees with Takaishi [29], who reports rough volatility for Bitcoin over an earlier and disjoint sample.

We report the block-bootstrap interval rather than the regression’s own standard error deliberately. The analytic standard error propagates only the scatter of the five $\zeta(q)$ about the fitted line; it ignores the sampling error inside each moment and the strong persistence of log-volatility, and on our data it is roughly an order of magnitude too small—it returns 2×10^{-4} for a window whose neighbouring window sits 0.02 away. Any inference drawn from it would be spurious.

Stability across the spot-ETF approval. The referee’s regime question has a natural event: the approval of US spot Bitcoin ETFs on 11 January 2024. We split the realised-volatility series into calendar 2023 (pre) and February–December 2024 (post), excluding January 2024 entirely as the event window rather than assigning it to either regime, and re-estimate:

$$H_{\mathbb{P}}^{\text{pre}} = 0.064 [0.018, 0.098], \quad H_{\mathbb{P}}^{\text{post}} = 0.062 [0.022, 0.079],$$

a difference of +0.002 against a combined bootstrap standard error of 0.025 ($z = 0.10$). The roughness of Bitcoin volatility is unchanged by the ETF approval. One caveat belongs with these numbers: this estimator is biased low on short windows, so the two one-year windows are comparable with each other but not with the full-sample figure above, and we compare only the former.

Option-implied, across the same split. The same event study can be run on option surfaces, with the caveat of Section 6.1: the historical panel is trade-based where the headline snapshot is quote-based. From it we build 52 pre-ETF (calendar 2023) and 47 post-ETF (February–December 2024) weekly surfaces. Because a single surface does not pin H , each week is fitted by Nelder–Mead from both ends of the ridge, never from the previous week, and Table 11 compares three things across the split: H within each basin, the identified parameters ρ and ν , and which basin the week prefers. The roughness does not move. From either warm start the mean weekly H is statistically indistinguishable before and after the approval (canonical start 0.099 against 0.107, bootstrap interval on the difference $[-0.071, +0.059]$; wide start 0.163 against 0.132, $[-0.059, +0.092]$), which agrees with the physical-measure result above from an entirely independent route. Two things do move: ν falls from 1.96 to 1.63 and the fit degrades from 124 to 151 vol bp, both with bootstrap intervals that exclude zero, and ρ shifts by -0.083 with an interval that only just excludes it. We read the last with care. The weekly ρ sits near zero, well away from the -0.45 of the headline snapshot, and that gap is the trade-versus-quote provenance difference of Section 6.1 rather than a property of the market: a comparison *across* weeks of the panel is sound, a comparison of its level with the headline fit is not. The weekly H itself varies widely from week to week and between the two starts on the same week, which is the non-identification of the headline surface reappearing at weekly resolution; the stability test is meaningful only because each regime is summarised over roughly fifty weeks.

Table 11: Option-implied parameters before and after the spot-ETF approval, 52 and 47 weekly surfaces. Mean $\pm$ SD per regime; difference pre–post with a 4-week block-bootstrap 95% interval and Welch p .

Quantity	pre-ETF	post-ETF	diff.	95% CI	p
H (canon. start)	0.099 ± 0.094	0.107 ± 0.133	-0.008	$[-0.071, +0.059]$	0.723
H (wide start)	0.163 ± 0.144	0.132 ± 0.154	$+0.031$	$[-0.059, +0.092]$	0.301
ρ	0.004 ± 0.167	0.086 ± 0.215	-0.083	$[-0.167, -0.003]$	0.037
ν	1.961 ± 0.570	1.631 ± 0.724	$+0.330$	$[+0.162, +0.689]$	0.014
fit (vol bp)	124.1 ± 47.3	150.6 ± 45.0	-26.5	$[-50.9, -15.8]$	0.005
basin gap (vol bp)	46.9 ± 83.8	43.4 ± 47.7	$+3.5$	$[-20.8, +33.6]$	0.797
Weeks preferring the smoother- H basin: 35% pre, 23% post; $n = 52$ and 47 weeks.					

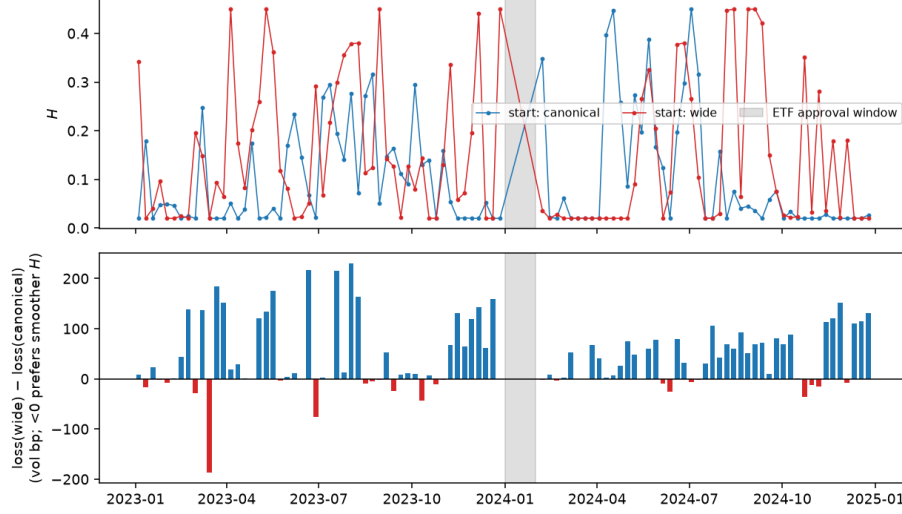


Figure 6: Weekly option-implied H from the trade-tape panel, fitted from both ends of the ridge (top), and which end each week prefers (bottom: loss from the wide-box start minus loss from the canonical start, negative preferring the smoother H). Shaded: January 2024, excluded as the ETF-approval window.

7.6. A parametric AI forward operator inside the inverse problem

A single network cannot cover both the breadth of a parameter box and the accuracy of a point-trained forward operator. Table 12 and Figure 8 report the calibration experiment with three seeds.

The parametric network adds $(H, \nu, \rho, \lambda, \theta)$ as five additional inputs, holding V_0 and the maturity fixed; we use $n=10$ (the crossover rule, Section 7.3) and width 96, depth 5. We train it on two boxes with identical architecture, budget and seeds: the original box of the submitted version ($\nu \leq 0.6$, $\theta \leq 0.35$) and a widened one ($\nu \leq 1.0$, $\theta \leq 0.45$) that contains the optimum of the $T=0.15$ slice. That turns the question of whether the box was mis-specified into a controlled ablation.

Solve-error depends strongly on where in parameter space it is measured, and the probe points have to be read with that in mind. At mild parameter vectors the network is accurate—3.6 to 9.1 px bp at the weak-skew and strong-skew probes—while at the rough, low- H probe it reaches 32–38 px bp. At the calibrated vector itself the widened box gives 9.1 ± 3.4 px bp against 33.8 ± 2.5 for the original; that vector has $\nu = 1.37$, outside *both* training boxes, so both figures are extrapolations and the widened box’s advantage

is that it extrapolates a shorter distance. The corners are quantified in Figure 7: the error concentrates where H is small and ν large, reaching 38–40 px bp, and it is an interaction rather than the sum of two effects—adding a ν/H term to an additive regression raises R^2 from 0.42–0.49 to 0.67–0.68, a partial R^2 of 0.19 to 0.25. That is the generalisation limit of this AI forward operator, stated explicitly rather than averaged away.

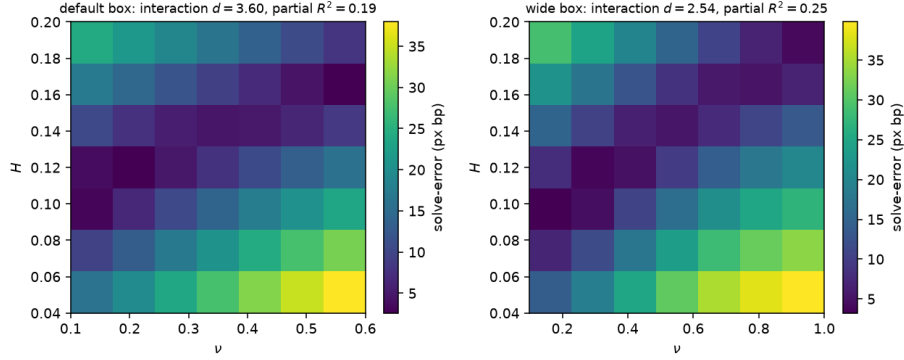


Figure 7: Parametric solve-error over (H, ν) at the box centre in (ρ, λ, θ) , $n=10$, seed-averaged, against a lifted Monte Carlo at every cell. Titles give the interaction coefficient d of $\text{err} \sim a + b/H + c\nu + d\nu/H$ and its partial R^2 .

Used as a standalone inverse-problem solver the parametric operator is fast (approximately 2s, zero true-model calls) but poor: 161 ± 15 volbp under the true model on the original box and 181 ± 97 on the widened one. The submitted version attributed this to the slice optimum lying outside the training box. The ablation does not support that explanation. The widened box contains the optimum ($\nu = 0.99$ against a bound of 1.0), yet its standalone fit is no better on average and far less reliable—the three seeds reach 316, 92 and 134 volbp. Containing the optimum is necessary but not sufficient: the network fits its own imperfect forward map rather than the market, and over a wider box that map is less accurate where it matters. We do not recommend the standalone parametric operator as an inverse-problem solver, on either box.

Used as a warm start it is effective, and on both boxes. Seeding a Nelder–Mead polish against the true forward model with the network’s parameter guess reaches 23.8 volbp from the original box and 23.6 from the widened one, against 22.7 for the from-scratch fit—on a slice whose market implied vols span roughly 34 to 52 per cent, so well inside a quoted spread. It does

so in 197 and 205 true-forward-model evaluations against 1 736, a saving of $8.8\times$ ($8.7\text{--}8.9\times$ across seeds) and $8.5\times$ respectively, and in 17 s against 142 s. The warm start is insensitive to which box the network was trained on, which is the useful practical conclusion: the hybrid does not require the training box to be chosen well. The value proposition is amortised: train the AI forward operator once, then solve each new instance of the inverse problem cheaply, with the exact forward model kept in the loop for a brief, decisive finish. The amortisation has a concrete break-even. Each warm-started recalibration saves about 1 535 true-model evaluations, or 125 s of wall-clock; training the parametric network costs 87 to 110 minutes on the widened and original boxes. The network pays for itself after 42 to 53 recalibrations of the same maturity slice: about two months of daily recalibration, or three and a half to four and a half hours of recalibration every five minutes. Below that, the exact forward model from scratch is the cheaper tool, and we would recommend it. The training times were measured while other jobs shared the machine, which inflates them, so the break-even is if anything overstated.

Table 12: Solving the calibration inverse problem for the BTC $T\approx 0.15$ slice: the true-forward-model loop from scratch, and the AI-only and hybrid routes with the parametric network trained on the original box and on a widened box that contains the slice optimum. Three network seeds each; fit quality is judged by the true Fourier forward model over the same feasible region.

Method	Fit RMSE (vol bp)	Wall-clock (s)	True-model evals
True-forward-model loop, from scratch	22.7	142	1736
AI-only (single-shot), original box	161 ± 15	~ 2	0
Hybrid (AI warm start, true-model polish), original box	23.8 ± 0.0	17	197 ± 1
AI-only (single-shot), widened box	181 ± 97	~ 2	0
Hybrid (AI warm start, true-model polish), widened box	23.6 ± 0.1	17	205 ± 1

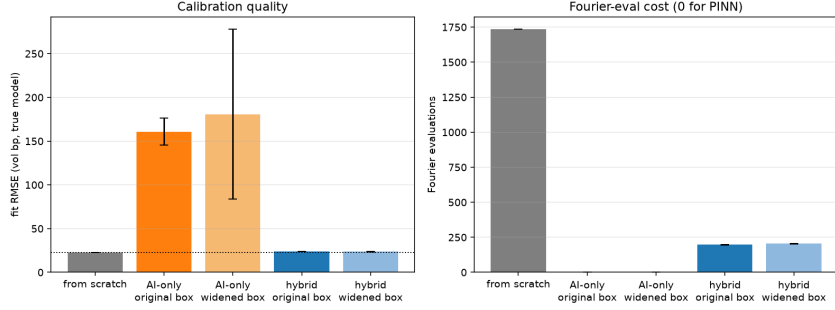


Figure 8: Hybrid solution of the inverse problem. Fit quality (left) and true-forward-model evaluation count (right) for the three methods. The hybrid matches from-scratch quality at $8.5\text{--}8.8\times$ fewer true-model evaluations whichever box the network was trained on; the AI-only route is fast but inaccurate, and widening the box to contain the optimum makes it no better on average and far less reliable across seeds.

7.7. Against the forward operators practitioners use

A reader who wants a fast forward operator for rough Heston would not reach for a PINN first. They would price with the Fourier transform directly, or run a short lifted Monte Carlo, or generate labels with an exact pricer and fit a network to them in the manner of Bayer et al. [13] and Horvath et al. [14]—the method class of the calibration networks of Liu et al. [30]. Table 13 places our operator next to all three under one protocol. The supervised surrogate is built to match the parametric network exactly: the same five inputs, the same fixed 21-point log-moneyness grid at $T=0.15$, the same widened box, Sobol-sampled and split by parameter vector so that no strike of a test vector is ever seen in training. We state the comparison plainly, because it goes against us on the axes most readers will check first. The supervised surrogate, trained on 64 353 Fourier-labelled parameter vectors, reaches 5.7 ± 0.1 vol bp against the transform method, and 9.3 ± 1.0 in the rough, high- ν corner where our parametric network degrades most. It is more accurate than the PINN against Fourier, and once trained it is as fast. On one-shot cost both the Fourier pricer and a short lifted Monte Carlo beat a PINN training run by three to four orders of magnitude. What the surrogate cannot do is exist without an exact pricer: every one of its labels came from the fractional Riccati solver, and in a model with no transform method it cannot be built at all. The PINN needs only the PDE. It also yields the solve/lift decomposition of Section 5.2, which a network fitted to Fourier labels inherits the lift error from silently and cannot separate. Those

two properties—no exact pricer offline, and an error budget split into what training controls and what the lift dimension controls—are the case for this operator, and we make no claim beyond them.

Table 13: Forward operators for the lifted rough-Heston smile at $T=0.15$. Accuracies are not comparable across rows, and the column says against what each is measured: the surrogate is trained on Fourier labels and scored against them, in implied-vol basis points, while the PINNs are scored in price basis points against an independent lifted Monte Carlo at the same n and carry the lift-error against Fourier on top. The parametric range runs from mild parameter vectors to the rough, high- ν corner. Both PINNs and the surrogate are differentiable in their inputs; the Fourier pricer and the Monte Carlo are not. Costs for the PINNs are CPU wall-clock; the parametric figure was measured on a shared machine and is indicative.

Operator	Offline cost	Exact pricer offline?	Accuracy (against)	Solve/lift split?
Fourier (fractional Riccati)	none	is one	exact by construction	no
Lifted MC, 10^6 paths	none	no	reference; SE 0.7–0.8 px bp	no
Single-point PINN, $n=4$	≈ 20 min per vector	no	15.2 px bp (lifted MC)	yes
Parametric PINN, $n=10$	80–125 min, once	no	3.6–38 px bp (lifted MC)	yes
Supervised surrogate (MLP)	71 min labels + 26 s	yes	5.7 vol bp (Fourier); 9.3 in the rough corner	no

8. Discussion

Convergence of the AI forward operator. The finding we care about most is that the network’s own error in solving the lifted forward PDE does not measurably degrade as the dimension grows, while grid methods become impractical as an inverse-problem engine. The history of that claim is worth recording, because it moved twice. An early single-seed run made the error look flat; three seeds suggested a doubling from $n=4$ to 32; eight seeds, a corrected reference and a formal test give a log-log slope of 0.09 ± 0.08 and no rejection of flatness ($p = 0.14$). We state the result in the form the eight-seed data support—no detectable growth, and growth that is in any case significantly sub-linear—and we note that it holds at the stiffer corrected

calibration, where the absolute error is also larger. The grid comparison uses the naive 50^{n+1} count; sparse-grid, ADI and tensor-train methods go further, so the defensible claim is *impractical, not impossible*. Even so, the contrast—an error and a cost that do not grow measurably with n , against a grid that cannot be stored—is a concrete instance of the mesh-free advantage as a component of an inverse-problem pipeline.

Stability: four design choices are load-bearing. We ablated each stabiliser explicitly. Without the hard-constrained ansatz, the soft IC penalty oscillates against the residual and the forward operator fails to converge. Without the integrated-variance anchor, a uniform 120 vol bp level bias remains and propagates directly into every downstream calibration. Without MC-informed factor sampling, the smile flattens and RMSE roughly doubles. Without self-adaptive weighting or the τ -curriculum, seed variance is markedly higher (from 339 ± 62 to 310 ± 28 vol bp in an early stability comparison). Each of these choices carries over to other AI forward operators for high-dimensional inverse problems.

A lesson about validating AI forward operators. The error decomposition revealed something methodologically important. When a learned surrogate replaces an exact forward solver inside an inverse problem, it is dangerously easy to validate it against the wrong reference. Our network solves the finite- n lifted PDE, whose exact solution is the lifted Monte Carlo, not the $n \rightarrow \infty$ true model; comparing straight to the true model conflates network error with model error. We found the two carry opposite signs and that the historical “network-versus-true-model” gap was lift-dominated, not a failure of the AI method. *Validate an AI forward operator against a same-fidelity reference, report the two errors separately, and let their crossover set the approximation order:* none of this is special to rough volatility, and we urge it on anyone placing a learned forward operator inside an inverse problem built on an approximated governing equation.

Generalisation and identifiability inside the inverse problem. A single network spread over five parameters is markedly less accurate than a point-trained one, worst in the rough, high-vol-of-vol corners where the PDE is stiffest—an explicit, quantified generalisation limit rather than an averaged headline number. Used alone as an inverse-problem solver it is additionally vulnerable to a training-box identifiability failure: if the true optimum lies outside the box the parametric operator was trained on, no amount of

gradient descent inside that box recovers it. This is not a limitation of the inverse problem itself, which remains well posed once regularised by the Fourier forward model; it is a limitation of trusting the AI forward operator unsupervised. The warm-start hybrid sidesteps exactly this failure mode by keeping the true forward model as the final arbiter.

Limitations and watch items. Our evidence rests on a single market snapshot (one Deribit BTC surface), and the parametric and calibration studies fix the maturity to a single slice. Timings are single-machine CPU figures at moderate budgets. The mild directional solve-bias at large n (ATM and call over-pricing growing to ~ 100 vol bp at $n=32$) is a watch item for high- n deployments inside a calibration loop. The sub-linear trend need not continue arbitrarily far.

Future work. Natural extensions include: (i) adding the maturity as a network input so a single forward operator serves the whole surface’s inverse problem at once; (ii) scaling width, depth, and GPU training to shrink the parametric corner penalty and widen the training box, directly addressing the identifiability failure mode above; (iii) extending to other rough models (rough Bergomi) and to American and path-dependent payoffs, where the inverse problem is harder still; (iv) investigating whether residual reweighting or a correction term resolves the high- n directional bias; (v) a formal stability/identifiability analysis of the calibration inverse problem itself as $n \rightarrow \infty$, complementing the purely empirical treatment given here.

9. Conclusion

We have characterised a physics-informed neural network as a forward operator for the rough-volatility calibration inverse problem, on the $(n+1)$ -dimensional Markovian-lifted PDE that makes the forward problem well-posed in the first place. Across an eight-fold rise in the lifted dimension the network’s solve-error shows no statistically detectable growth and its training cost per iteration is flat, while naive finite-difference forward solvers become infeasible by $n=8$; we identify the four training choices that make this behaviour hold, and show that the held-out PDE residual is equally flat, so what variation remains lies in the stability constant of the lifted operator rather than in the optimiser. Separating solve-error from lift-error is essential to correctly attribute inverse-problem error: it told us to add factors at one calibration and to improve the network at another, and it yields a

design rule for the factor count that holds in both. Applied to live Bitcoin options, the surface pins the correlation but bounds the Hurst exponent only to $[0.04, 0.30]$ —rough, but not a point—and that ambiguity moves the variance-swap strike and the risk reversal by well under a vol point. A parametric extension of the forward operator, while not accurate enough to solve the inverse problem independently, is an effective warm start that solves it at 8.8 times fewer expensive forward-model evaluations. The methodological message is that an AI forward operator earns its place inside an inverse-problem pipeline only once its own error is separated from the model error it is approximating—and that, done this way, it degrades gracefully where grid methods cannot follow.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Data availability statement

The BTC option data were retrieved from the Deribit public REST API (no API key required) at <https://www.deribit.com/api/v2>. The raw data snapshot used in the experiments, the calibrated parameter files, the two robustness datasets of Section 6.1 in derived form (weekly implied-vol surfaces and daily realised variance), every result file the tables are generated from, and the complete PyTorch implementation are available at <https://github.com/aviral9876/physics-informed-nn-rough-vol>.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this manuscript the authors used Claude (Anthropic) to assist with prose drafting and structural organisation. After using this tool, the authors reviewed and edited all content as needed and take

full responsibility for the accuracy and integrity of the published work. The mathematical derivations, experimental results, code, and all scientific claims are the authors' own.

Appendix A. Derivation of the lifted pricing PDE

We derive (4) from the lifted dynamics (3), since a referee correctly observed that the correspondence is not self-evident — in particular the presence of a *full* (i, j) double sum in the factor block, where a reader expecting independent Ornstein–Uhlenbeck factors would predict a diagonal one.

Collect the state as $Z_t = (X_t, U_t^1, \dots, U_t^n)$ with $X_t = \log S_t$. Under the risk-neutral measure the lifted system is

$$dX_t = \left(r - \frac{1}{2}V_t\right) dt + \sqrt{V_t^+} dB_t, \quad (\text{A.1})$$

$$dU_t^i = \left[-x_i U_t^i + \lambda(\theta - V_t)\right] dt + \nu \sqrt{V_t^+} dW_t^V, \quad i = 1, \dots, n, \quad (\text{A.2})$$

with $V_t = V_0 + \sum_i c_i U_t^i$ and $d\langle B, W^V \rangle_t = \rho dt$. This is a $(n+1)$ -dimensional Itô diffusion, hence Markovian — which is the entire purpose of the lift, the original Volterra variance process being neither.

The instantaneous covariation matrix of Z follows from the fact that the system is driven by only *two* Brownian motions, B and W^V , whatever the value of n :

$$\begin{aligned} \frac{d\langle X, X \rangle_t}{dt} &= V_t^+, & \frac{d\langle X, U^i \rangle_t}{dt} &= \rho \nu V_t^+, \\ \frac{d\langle U^i, U^j \rangle_t}{dt} &= \nu^2 V_t^+ \quad \text{for all } i, j. \end{aligned} \quad (\text{A.3})$$

The last identity is the crux. Every factor is driven by the same increment dW_t^V and differs only through its mean-reversion speed x_i , so the factor block of the covariation matrix is $\nu^2 V_t^+ \mathbf{1}\mathbf{1}^\top$ — rank one, with every entry equal, rather than diagonal. The factors are perfectly instantaneously correlated; they decorrelate only through their differing decay rates over finite time. Consequently the second-order factor term in the generator carries the unrestricted double sum $\sum_{i,j=1}^n P_{u_i u_j}$ that appears in (4), and not $\sum_i P_{u_i u_i}$.

The infinitesimal generator of Z is therefore

$$\begin{aligned} \mathcal{A}P = & \left(r - \frac{1}{2}V\right) P_x + \sum_{i=1}^n [-x_i u_i + \lambda(\theta - V)] P_{u_i} \\ & + \frac{1}{2}V P_{xx} + \frac{1}{2}\nu^2 V \sum_{i,j=1}^n P_{u_i u_j} + \rho\nu V \sum_{i=1}^n P_{xu_i}, \end{aligned} \quad (\text{A.4})$$

where we write V for V^+ to lighten notation, the truncation being active only on a set the scheme controls numerically. For a European claim with payoff $g(X_T)$, the discounted price $P(t, x, u) = \mathbb{E}[e^{-r(T-t)}g(X_T) \mid Z_t = (x, u)]$ satisfies the Feynman–Kac equation $P_t + \mathcal{A}P - rP = 0$ with $P(T, x, u) = g(x)$, which on substituting $\mathcal{A}$ and $g(x) = \max(e^x - K, 0)$ is exactly (4).

Three remarks bear on the numerics. First, the drift source $\lambda(\theta - V)$ and the diffusion coefficient $\nu\sqrt{V}$ are *shared* across factors; only $-x_i u_i$ distinguishes them, which is why the factors can be advanced with a single common Brownian draw. Second, the (x_i) span roughly ten orders of magnitude at small H , making the factor block stiff and forcing the exponential-integrator treatment of Section 5. Third, the rank-one structure of the factor covariation is what couples ρ and ν across the whole factor block, and is the reason the low-rank tensor structure that a tensor-train solver would need is eroded rather than preserved as n grows.

References

- [1] J. Gatheral, T. Jaisson, M. Rosenbaum, Volatility is rough, *Quantitative Finance* 18 (6) (2018) 933–949. doi:10.1080/14697688.2017.1393551.
- [2] M. Bennedsen, A. Lunde, M. S. Pakkanen, Decoupling the short- and long-term behavior of stochastic volatility, *Journal of Financial Econometrics* 20 (5) (2022) 961–1006. doi:10.1093/jjfinec/nbaa049.
- [3] S. L. Heston, A closed-form solution for options with stochastic volatility with applications to bond and currency options, *Review of Financial Studies* 6 (2) (1993) 327–343. doi:10.1093/rfs/6.2.327.
- [4] F. Comte, E. Renault, Long memory in continuous-time stochastic volatility models, *Mathematical Finance* 8 (4) (1998) 291–323. doi:10.1111/1467-9965.00057.

- [5] O. El Euch, M. Rosenbaum, The characteristic function of rough Heston models, *Mathematical Finance* 29 (1) (2019) 3–38. doi:10.1111/mafi.12173.
- [6] E. Abi Jaber, O. El Euch, Multifactor approximation of rough volatility models, *SIAM Journal on Financial Mathematics* 10 (2) (2019) 309–349. doi:10.1137/18M1170236.
- [7] E. Abi Jaber, Lifting the Heston model, *Quantitative Finance* 19 (12) (2019) 1995–2013. doi:10.1080/14697688.2019.1615113.
- [8] M. Raissi, P. Perdikaris, G. E. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, *Journal of Computational Physics* 378 (2019) 686–707. doi:10.1016/j.jcp.2018.10.045.
- [9] J. Sirignano, K. Spiliopoulos, DGM: A deep learning algorithm for solving partial differential equations, *Journal of Computational Physics* 375 (2018) 1339–1364. doi:10.1016/j.jcp.2018.08.029.
- [10] J. Han, A. Jentzen, W. E, Solving high-dimensional partial differential equations using deep learning, *Proceedings of the National Academy of Sciences* 115 (34) (2018) 8505–8510. doi:10.1073/pnas.1718942115.
- [11] A. Papapantoleon, J. Rou, A time-stepping deep gradient flow method for option pricing in (rough) diffusion models, *Quantitative Finance* 25 (12) (2025) 2009–2020. doi:10.1080/14697688.2025.2572318.
- [12] A. Jacquier, v. Zurič, Random neural networks for rough volatility, preprint, arXiv:2305.01035 (2023). arXiv:2305.01035.
- [13] C. Bayer, B. Horvath, A. Muguruza, B. Stemper, M. Tomas, On deep calibration of (rough) stochastic volatility models, preprint, arXiv:1908.08806 (2019). arXiv:1908.08806.
- [14] B. Horvath, A. Muguruza, M. Tomas, Deep learning volatility: a deep neural network perspective on pricing and calibration in (rough) volatility models, *Quantitative Finance* 21 (1) (2021) 11–27. doi:10.1080/14697688.2020.1817974.

- [15] A. S. Krishnapriyan, A. Gholami, S. Zhe, R. M. Kirby, M. W. Mahoney, Characterizing possible failure modes in physics-informed neural networks, *Advances in Neural Information Processing Systems (NeurIPS)* 34 (2021) 26548–26560.
- [16] S. Wang, X. Yu, P. Perdikaris, When and why PINNs fail to train: a neural tangent kernel perspective, *Journal of Computational Physics* 449 (2022) 110768. doi:10.1016/j.jcp.2021.110768.
- [17] A. Kendall, Y. Gal, R. Cipolla, Multi-task learning using uncertainty to weigh losses for scene geometry and semantics, *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2018) 7482–7491 doi:10.1109/CVPR.2018.00781.
- [18] L. D. McClenny, U. M. Braga-Neto, Self-adaptive physics-informed neural networks, *Journal of Computational Physics* 474 (2023) 111722. doi:10.1016/j.jcp.2022.111722.
- [19] S. Wang, S. Sankaran, P. Perdikaris, Respecting causality for training physics-informed neural networks, *Computer Methods in Applied Mechanics and Engineering* 421 (2024) 116813. doi:10.1016/j.cma.2024.116813.
- [20] P. Carr, D. B. Madan, Option valuation using the fast Fourier transform, *Journal of Computational Finance* 2 (4) (1999) 61–73. doi:10.21314/JCF.1999.043.
- [21] X. Glorot, Y. Bengio, Understanding the difficulty of training deep feed-forward neural networks, in: *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2010, pp. 249–256.
- [22] D. P. Kingma, J. Ba, Adam: a method for stochastic optimization, *International Conference on Learning Representations (ICLR)* ArXiv:1412.6980 (2015).
- [23] H.-J. Bungartz, M. Griebel, Sparse grids, *Acta Numerica* 13 (2004) 147–269.

- [24] C. Reisinger, G. Wittum, Efficient hierarchical approximation of high-dimensional option pricing problems, *SIAM Journal on Scientific Computing* 29 (1) (2007) 440–458.
- [25] K. J. in 't Hout, S. Foulon, ADI finite difference schemes for option pricing in the Heston model with correlation, *International Journal of Numerical Analysis and Modeling* 7 (2) (2010) 303–320.
- [26] I. V. Oseledets, Tensor-train decomposition, *SIAM Journal on Scientific Computing* 33 (5) (2011) 2295–2317.
- [27] S. Mishra, R. Molinaro, Estimates on the generalization error of physics-informed neural networks for approximating PDEs, *IMA Journal of Numerical Analysis* 43 (1) (2023) 1–43.
- [28] T. De Ryck, S. Mishra, Error analysis for physics-informed neural networks (PINNs) approximating Kolmogorov PDEs, *Advances in Computational Mathematics* 50 (4) (2024) 83.
- [29] T. Takaishi, Rough volatility of Bitcoin, *Finance Research Letters* 32 (2020) 101379.
- [30] S. Liu, A. Borovykh, L. A. Grzelak, C. W. Oosterlee, A neural network-based framework for financial model calibration, *Journal of Mathematics in Industry* 9 (1) (2019) 9.